

GUJARAT TECHNOLOGICAL UNIVERSITY

Chandkheda, Ahmedabad

Affiliated



Sardar Vallabhbhai Patel Institute of Technology, Vasad

A

Report On

(Self-Learning Activity: 15-Hour Software Development Project)

Project Title: Personal Movie/Media Vault

Under the subject of

Object Oriented Programming (BE04000231)

BE, Year 2, Semester 4

(Computer Engineering)

Submitted by:

Enrollment No.	Name
240410107057	THAKKAR KARTIK PRAMODKUMAR

Prof. Keyur Suthar
(Faculty Guide)

Prof. Jayna Shah
(Head of Department)

Academic year (2026)

INDEX

Sr no.	Table of Contents
1	Project Overview
2	Objectives
3	Object-Oriented Programming Concepts Used
4	Project Structure
5	How to Compile and Run
6	Sample Output
7	Conclusion
8	Complete Source Code
9	Example of Output

1. Project Overview

The **Personal Movie/Media Vault** is a Command Line Interface (CLI)-based Java application designed to help users manage their personal movie collection. The system allows users to log in, add movies, rate them, filter by genre, and get recommendations.

Each user has a separate file for storing their movie data, ensuring personalized and organized data management. The project demonstrates the use of file handling and Object-Oriented Programming (OOP) concepts.

2. Objectives

- To develop a CLI-based Java application
- To implement file handling for data storage
- To apply Object-Oriented Programming concepts
- To manage user-specific movie records
- To provide a basic recommendation system

3. Object-Oriented Programming Concepts Used

3.1 Inheritance

The class hierarchy **Movie** → **UserRating** forms a simple inheritance structure. **Movie** is the base class that stores common attributes such as title, year, genre, and rating. **UserRating** extends it by adding user-specific fields like username and review.

This allows reuse of core movie properties while extending functionality for personalized data handling without duplicating code.

3.2 Abstraction via Classes

Abstraction is achieved by encapsulating complex logic inside classes such as **MediaVault** and **LoginCLI**.

The **MediaVault** class abstracts operations like saving and loading data from files, while **LoginCLI** abstracts the authentication process.

The user interacts only with a simple CLI menu, without needing to understand internal implementation details.

3.3 Encapsulation

All important fields in classes are declared as private, and access is controlled through public methods.

For example, in the **UserRating** class, the username field is private and accessed via a getter method.

This ensures that internal data is protected and cannot be modified directly from outside the class.

SUB NAME: Object Oriented Programming

SUBJECT CODE:BE04000231

Additionally, operations like adding movies are handled through controlled methods in MediaVault, ensuring that data is always stored correctly and consistently.

3.4 Polymorphism

Polymorphism is implemented through method overriding.

The toString() method is overridden in the UserRating class to provide additional user-specific details while still using the base class representation from Movie.

This allows the same method call (toString()) to produce different outputs depending on the object type, improving flexibility and readability.

3.5 Exception Handling

The project uses exception handling to manage invalid inputs and file-related errors.

For example, invalid ratings (outside the allowed range) are handled using custom or runtime exceptions.

File operations such as reading and writing are enclosed within try-catch blocks to prevent program crashes.

This ensures robustness and smooth execution of the application even in error scenarios.

3.6 File I/O (Data Persistence)

The application uses file handling to store and retrieve user data persistently.

Each user has a separate file (e.g., data/username.txt) where their movie records are stored. The MediaVault class manages all file operations using FileWriter and BufferedReader.

Data is automatically saved whenever a movie is added and loaded when the user logs in, ensuring that user data is retained across program executions.

3.7 Modularity (Separation of Concerns)

The system is divided into multiple classes, each handling a specific responsibility:

- LoginCLI handles authentication
- MediaVault manages movie data and file operations
- Recommender handles recommendation logic
- UserRating represents user-specific movie data

This modular design improves maintainability, readability, and scalability of the application.

4.Project Structure

Package / File	Responsibility
MainCLI.java	Controls the main program flow, displays menu, and handles user interaction
LoginCLI.java	Manages user authentication by verifying username and password from file
MediaVault.java	Handles core logic like adding movies, loading/saving data, filtering, and displaying movies
UserRating.java	Stores user-specific movie details including username and review
Movie.java	Base class containing common movie attributes like title, year, genre, and rating
Recommender.java	Provides movie recommendations based on user's preferred genre
data/users.txt	Stores registered usernames and passwords
data/<username>.txt	Stores movie data for each individual user separately
data/ folder	Contains all user-related data files
InvalidRatingException	Custom exception to handle invalid rating input

SUB NAME: Object Oriented Programming

SUBJECT CODE:BE04000231

5. How to Compile and Run

Step 1: Open Terminal

cd YourProject

Ex: cd CLI or cd Project

Step 2: Compile

javac src/*.java

Step 3: Run

java -cp src MainCLI

6. Sample Output

Username: Kartik

Password: ****

Login Successful!

-----Welcome To The Personal Movie Vault-----

1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit

Enter your choice: 1

Movie Title: Avengers

Year: 2020

Genre: Action

Rating: 4.5

Review: Great movie

7. Conclusion

The project successfully demonstrates the implementation of a CLI-based system using Java. It effectively uses OOP concepts and file handling to manage user-specific movie data. The system is simple, efficient, and can be extended with advanced features like GUI or database integration.

SUB NAME: Object Oriented Programming

SUBJECT CODE:BE04000231

8. Complete Source Code

Project/

```
|
|
|— src/
|   |— MainCLI.java
|   |— LoginCLI.java
|   |— MediaVault.java
|   |— UserRating.java
|   |— Movie.java
|   |— Recommender.java
|   |— InvalidRatingException.java
|
|
|— data/
|   |— users.txt
|   |— Kartik.txt
|   |— Rahul.txt
```

File : Project/src/MainCLI.java

Role : Main controller of the application

Program :

```
import java.util.*;

public class MainCLI {

    public static void main(String[] args) {

        LoginCLI login = new LoginCLI();
        if (!login.login()) return;

        Scanner sc = new Scanner(System.in);
        MediaVault vault = new MediaVault();
        Recommender rec = new Recommender();

        vault.loadFromFile(LoginCLI.currentUser);

        System.out.println("-----Welcome To The Personal Movie/Media Vault App-----");

        while (true) {
            System.out.println("\n1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show
6.Exit");
            System.out.print("Enter your choice: ");
            int ch = sc.nextInt();
            sc.nextLine();
            try {
                switch (ch) {
                    case 1:
                        System.out.print("Movie Title: ");
                        String t = sc.nextLine();

                        System.out.print("Year: ");
                        int y = sc.nextInt(); sc.nextLine();

                        System.out.print("Genre: ");
                        String g = sc.nextLine();
```

```
System.out.print("Rating: ");
double r = sc.nextDouble(); sc.nextLine();

System.out.print("Review: ");
String rev = sc.nextLine();

if (r < 0 || r > 10)
    throw new InvalidRatingException("Invalid Rating!");

vault.addMovie(new UserRating(
    LoginCLI.currentUser,
    t, y, g, r, rev
));
break;

case 2:
    vault.getTopRated();
    break;

case 3:
    System.out.print("Genre: ");
    vault.filterByGenre(sc.nextLine());
    break;

case 4:
    System.out.print("Fav Genre: ");
    rec.recommend(vault.getMovies(), sc.nextLine());
    break;

case 5:
    vault.showUserMovies();
    break;

case 6:
    System.out.println("bye see you agian!");
    return; }
```

SUB NAME: Object Oriented Programming

SUBJECT CODE:BE04000231

```
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
}  
}  
}
```

File : Project/src/ LoginCLI.java

Role : User authentication system

Program :

```
import java.io.*;
import java.util.*;

class LoginCLI {
    public static String currentUser;
    boolean login() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Username: ");
        String u = sc.nextLine().trim();
        System.out.print("Password: ");
        String p = sc.nextLine().trim();
        try {
            BufferedReader br = new BufferedReader(new FileReader("data/users.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                String[] d = line.split(",");
                if (d[0].equalsIgnoreCase(u) && d[1].equals(p)) {
                    currentUser = u;
                    System.out.println("Login Successful!");
                    br.close();
                    return true;
                }
            }
            br.close();
        } catch (Exception e) {
            System.out.println("Error reading users file!");
        }
        System.out.println("Invalid Login!");
        return false;
    }
}
```

File : Project/src/ MediaVault.java

Role : Core logic and data management

Program :

```
import java.io.*;
import java.util.*;

class MediaVault {
    private List<UserRating> movies = new ArrayList<>();

    public void addMovie(UserRating m) {
        movies.add(m);
        saveToFile(m);
    }

    public List<UserRating> getMovies() {
        return movies;
    }

    public void filterByGenre(String genre) {
        for (UserRating m : movies) {
            if (m.getGenre().equalsIgnoreCase(genre)) {
                System.out.println(m);
            }
        }
    }

    public void getTopRated() {
        movies.stream()
            .sorted((a, b) -> Double.compare(b.getRating(), a.getRating()))
            .limit(5)
            .forEach(System.out::println);
    }
}
```


SUB NAME: Object Oriented Programming

SUBJECT CODE:BE04000231

```
Integer.parseInt(parts[2]),
    parts[3],
    Double.parseDouble(parts[4]),
    parts[5]
));
}

br.close();

} catch (Exception e) {
    System.out.println("No data found for this user.");
}
}

public void showUserMovies() {
    if (movies.isEmpty()) {
        System.out.println("No data found.");
    } else {
        for (UserRating movie : movies) {
            System.out.println(movie);
        }
    }
}
}
```

File : Project/src/ UserRating.java

Role : User-specific movie data model

Program :

```
class UserRating extends Movie {  
    private String review;  
    private String username;  
  
    public UserRating(String username, String title, int year, String genre, double rating, String review)  
    {  
        super(title, year, genre, rating);  
        this.username = username;  
        this.review = review;  
    }  
  
    public String getUsername() {  
        return username;  
    }  
  
    public String getReview() {  
        return review;  
    }  
  
    public String toString() {  
        return username + " -> " + super.toString() + " | Review: " + review;  
    }  
  
    public String toFileString() {  
        return username + "," + title + "," + year + "," + genre + "," + rating + "," + review;  
    }  
}
```

File : Project/src/ Movie.java**Role : Base class for movie details****Program :**

```
class Movie {  
    protected String title;  
    protected int year;  
    protected String genre;  
    protected double rating;  
  
    public Movie(String title, int year, String genre, double rating) {  
        this.title = title;  
        this.year = year;  
        this.genre = genre;  
        this.rating = rating;  
    }  
  
    public String getTitle() { return title; }  
    public int getYear() { return year; }  
    public String getGenre() { return genre; }  
    public double getRating() { return rating; }  
  
    public String toString() {  
        return title + " (" + year + ") | " + genre + " | Rating: " + rating;  
    }  
}
```

File : Project/src/ Recommender.java

Role : Recommendation system

Program :

```
import java.util.List;
```

```
class Recommender {

    public void recommend(List<UserRating> movies, String genre) {
        UserRating best = null;

        for (UserRating m : movies) {
            if (m.getGenre().equalsIgnoreCase(genre)) {
                if (best == null || m.getRating() > best.getRating()) {
                    best = m;
                }
            }
        }

        if (best != null)
            System.out.println("Recommended:\n" + best);
        else
            System.out.println("No recommendation found.");
    }
}
```

File : Project/src/ InvalidRatingException.java

Role : Custom exception handling

Program :

```
class InvalidRatingException extends Exception {  
    public InvalidRatingException(String msg) {  
        super(msg);  
    }  
}
```

File : Project/src/ data/users.txt

Role : User credentials storage

Program :

admin,123

kartik,Kt@123

9.Example of Output

```
PS C:\Users\Thakkar kartik\Desktop\java lab> cd Project
PS C:\Users\Thakkar kartik\Desktop\java lab\Project> javac src/*.java
PS C:\Users\Thakkar kartik\Desktop\java lab\Project> java -cp src MainCLI
Username: kartik
Password: Kt@123
Login Successful!
No data found for this user.
-----Welcome To The Personal Movie/Media Vault App-----

1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 1
Movie Title: Avengers
Year: 2020
Genre: Action
Rating: 4.5
Review: Great Movie

1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 1
Movie Title: Hi papa
Year: 2024
Genre: Emotional
Rating: 5
Review: Excellent
```

```
PS C:\Users\Thakkar kartik\Desktop\java lab> cd Project
PS C:\Users\Thakkar kartik\Desktop\java lab\Project> javac src/*.java
PS C:\Users\Thakkar kartik\Desktop\java lab\Project> java -cp src MainCLI
Username: kartik
Password: Kt@123
Login Successful!
-----Welcome To The Personal Movie/Media Vault App-----
```

```
1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 2
kartik -> Hi Papa (2024) | Emotional | Rating: 5.0 | Review: Excellent
kartik -> Avengers (2020) | Action | Rating: 4.5 | Review: Great movie
```

```
1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 3
Genre: Emotional
kartik -> Hi Papa (2024) | Emotional | Rating: 5.0 | Review: Excellent
```

```
1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 4
Fav Genre: Action
Recommended:
kartik -> Avengers (2020) | Action | Rating: 4.5 | Review: Great movie
```

```
1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 5
kartik -> Avengers (2020) | Action | Rating: 4.5 | Review: Great movie
kartik -> Hi Papa (2024) | Emotional | Rating: 5.0 | Review: Excellent
```

```
1.AddMovie 2.TopRated 3.FilterByGenre 4.Recommend 5.Show 6.Exit
Enter your choice: 6
bye see you agian!
PS C:\Users\Thakkar kartik\Desktop\java lab\Project>
```